

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Error Analysis Task

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.* (BL5)

Assessment Tool Weightage: 20%

Dave's Taxonomy: Articulation (Level 4)

Total Marks: 25

Question Count: 5

Code of Conduct

I Beffy A Shanika(192511387) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: beffy

Assessment Tasks

Error Analysis Task

1. A Hadoop job repeatedly fails because input splits are uneven and reducers remain idle. Identify the likely design error and propose corrective actions.
2. An HDFS deployment shows frequent data unavailability after a single node failure. Analyze the probable replication or NameNode design error.
3. A data ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics store. Identify the likely causes and recommend fixes.
4. A YARN cluster suffers from resource starvation when multiple users submit jobs together. Analyze the scheduling error and suggest a better resource management approach.
5. A backup strategy for distributed storage restores outdated data after recovery. Identify the probable design flaw in the replication or backup approach.

Error Analysis Task Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Identification of Error	30%	Accurately identifies the root technical issue	Identifies most of the issue	Partially identifies issue	Fails to identify issue
Cause Analysis	20%	Explains root cause with strong technical reasoning	Reasonable cause analysis	Basic cause analysis	Weak analysis
Corrective Action	25%	Proposes effective and feasible corrections	Reasonable corrections	Basic corrections	Ineffective corrections
Use of Hadoop / Integration Concepts	15%	Applies relevant concepts appropriately	Mostly appropriate application	Partial application	Incorrect application
Clarity	10%	Clear and direct explanation	Generally clear	Somewhat unclear	Poorly presented

ANSWER

1. Uneven Input Splits and Idle Reducers

Problem:

The Hadoop job fails repeatedly because input splits are uneven and some reducers remain idle.

Likely Design Error:

- Improper input split/block size.
- Presence of very large or very small files.
- Poor partitioning of mapper output.
- Incorrect number of reducers.

Corrective Actions:

- Configure an appropriate HDFS block size.
- Combine small files before processing.

- Use a suitable InputFormat to create balanced splits.
- Use a proper partitioner to distribute mapper output evenly.
- Adjust the number of reducers according to data volume.
- Use a Combiner where applicable to reduce intermediate data.

Conclusion:

Balanced input splits and proper partitioning ensure that mapper and reducer tasks are utilized efficiently.

2. HDFS Data Unavailability After One Node Failure

Problem:

Data becomes unavailable when a single DataNode fails.

Likely Design Error:

- Replication factor may be 1 or too low.
- Replicas may not be distributed across different DataNodes/racks.
- NameNode may have no High Availability (HA) configuration.
- HDFS may not have completed block replication.

Corrective Actions:

- Set the replication factor to 3 for critical data.
- Use rack-aware replica placement.
- Configure Active and Standby NameNodes.
- Monitor under-replicated blocks.
- Enable automatic re-replication when a DataNode fails.

Conclusion:

Proper replication and NameNode HA ensure that failure of one node does not make data unavailable.

3. Duplicate Records in ETL and Apache NiFi

Problem:

The ingestion workflow creates duplicate records in the analytics store.

Likely Causes:

- The same file or record is processed multiple times.
- NiFi flow does not maintain proper state/checkpoints.
- ETL pipeline is not idempotent.
- Network retries cause the same data to be inserted again.
- No unique key or deduplication mechanism exists.
- CDC/incremental ingestion is incorrectly configured.

Corrective Actions:

- Use unique record IDs or business keys.
- Implement deduplication using keys or hashes.
- Configure NiFi state management/checkpointing.
- Use incremental/CDC-based ingestion.
- Make database writes idempotent using upsert/merge operations.
- Track successfully processed files and records.

Conclusion:

State tracking, unique identifiers and idempotent processing prevent duplicate data during retries or repeated ingestion.

4. YARN Resource Starvation

Problem:

Multiple users submit jobs and some jobs receive insufficient resources or remain waiting for a long time.

Likely Scheduling Error:

- No proper queue configuration.
- One user/job consumes most cluster resources.
- Incorrect resource limits.
- Poor scheduler configuration.
- No fairness mechanism.

Better Approach:

- Use Capacity Scheduler or Fair Scheduler.
- Create separate queues for different users or workloads.
- Set minimum and maximum resource capacities for queues.
- Apply per-user and per-application resource limits.
- Enable preemption when appropriate.
- Monitor CPU, memory and container utilization.

Example:

YARN

|— ETL Queue → 40%

|— Analytics Queue → 30%

└— ML Queue → 30%

Conclusion:

Queue-based resource allocation and fair scheduling prevent a single user or job from consuming all cluster resources.

5. Outdated Data Restored After Backup Recovery

Problem:

After a failure, the backup restores an older version of the data.

Likely Design Flaw:

- Backups are performed too infrequently.
- Backup copies are not synchronized with current data.
- Only full backups are used with large intervals.
- Replication is being treated as a backup without version/history protection.
- No verification of the latest successful backup.

Corrective Actions:

- Use frequent incremental backups along with periodic full backups.
- Maintain multiple backup versions/snapshots.
- Use timestamps and versioning to identify the latest valid copy.
- Monitor and verify successful backup completion.
- Keep backups in a separate failure domain.
- Regularly perform restore testing.

Conclusion:

A combination of frequent incremental backups, versioned snapshots and backup verification ensures that recovery uses the latest valid data rather than an outdated copy.